CONCEPTOS DEL DÍA · SESIÓN 22

Paso de parámetros por valor y por dirección

Documento de consulta para la persona estudiante

Módulo	CSTI12010 Diseño de algoritmos
Unidad de aprendizaje	UA2 · Empleo de lenguajes de programación en el paradigma de programación estructurada
Capacidad	2.3 Emplear funciones en la resolución de problemas según las especificaciones técnicas.
Saberes	Transferencia de parámetros por valor · Transferencia de parámetros por dirección de memoria (punteros) · Arreglos como parámetros · Integración: funciones de caracteres, numéricas y de cadenas
Persona facilitadora	Giovanni Antonio Coto Calderón

Para qué le sirve este documento

Resume y amplía lo visto en la presentación. Úselo para repasar, hacer la práctica autónoma y prepararse para la Actividad de comprobación 2. Al final encontrará un glosario y preguntas de repaso.

1. Paso por valor

El parámetro recibe una **copia**: la función no puede cambiar la variable original.

valor.c

```c
#include <stdio.h>

void duplicar(int x) {
    x = x * 2;                  // cambia la COPIA
    printf("dentro: x = %d\n", x);
}

int main(void) {
    int numero = 5;
    duplicar(numero);
    printf("main: numero = %d\n", numero);
    return 0;
}
```

Salida real (gcc)

```
dentro: x = 10
main: numero = 5
```

2. Direcciones y punteros

Símbolo	Significado	Ejemplo
`&x`	Dirección de x	`scanf("%d", &x);`
`int *p`	p guarda la dirección de un int	`int *p = &x;`
`*p`	Valor en esa dirección	`*p = 21;`

puntero.c

```c
#include <stdio.h>

int main(void) {
    int edad = 20;
    int *p = &edad;             // p guarda la direccion
    printf("edad = %d\n", edad);
    printf("*p   = %d\n", *p);  // lee lo que hay alli
    *p = 21;                    // cambia edad a traves de p
    printf("edad = %d\n", edad);
    return 0;
}
```

Salida real (gcc)

```
edad = 20
*p   = 20
edad = 21
```

3. Paso por dirección

intercambio.c

```c
#include <stdio.h>

void intercambiar(int *a, int *b) {
    int aux = *a;
    *a = *b;
    *b = aux;
}

int main(void) {
    int x = 5, y = 9;
    printf("Antes:   x = %d  y = %d\n", x, y);
    intercambiar(&x, &y);       // envia direcciones
    printf("Despues: x = %d  y = %d\n", x, y);
    return 0;
}
```

Salida real (gcc)

```
Antes:   x = 5  y = 9
Despues: x = 9  y = 5
```

dividir.c

```c
#include <stdio.h>

void dividir(int a, int b, int *cociente, int *residuo) {
    *cociente = a / b;
    *residuo = a % b;
}

int main(void) {
    int c, r;
    dividir(17, 5, &c, &r);     // dos resultados
    printf("17 / 5 = %d residuo %d\n", c, r);
    return 0;
}
```

Salida real (gcc)

```
17 / 5 = 3 residuo 2
```

4. Arreglos y cadenas como parámetros

Un arreglo se pasa por su nombre (ya es una dirección) junto con su tamaño; **const** impide modificarlo. Una cadena se recorre hasta el **'\0'**.

arreglos.c

```c
#include <stdio.h>
#define N 5

double promedio(const double notas[], int n) {
    double suma = 0;
    int i;
    for (i = 0; i < n; i++) { suma += notas[i]; }
    return suma / n;
}

void aplicarCurva(double notas[], int n, double puntos) {
    int i;
    for (i = 0; i < n; i++) { notas[i] += puntos; }
}

int main(void) {
    double notas[N] = {80, 75, 90, 65, 100};
    printf("Promedio: %.2f\n", promedio(notas, N));
    aplicarCurva(notas, N, 2);  // cambia el original
    printf("Con curva: %.2f\n", promedio(notas, N));
    return 0;
}
```

Salida real (gcc)

```
Promedio: 82.00
Con curva: 84.00
```

vocales.c

```c
#include <stdio.h>
#include <ctype.h>

int contarVocales(const char texto[]) {
    int i, total = 0;
    char c;
    for (i = 0; texto[i] != '\0'; i++) {
        c = tolower(texto[i]);
        if (c == 'a' || c == 'e' || c == 'i' ||
            c == 'o' || c == 'u') {
            total++;
        }
    }
    return total;
}

int main(void) {
    printf("Vocales: %d\n", contarVocales("Instituto Nacional"));
    return 0;
}
```

Salida real (gcc)

```
Vocales: 8
```

5. Estructura de un programa modular

1. Comentario inicial: qué se logra y resultado esperado.
2. #include de las bibliotecas.
3. #define de las constantes.
4. Prototipos de las funciones.
5. main con el menú o el flujo principal.
6. Definiciones de las funciones, una tarea cada una.

6. Errores frecuentes

Error	Qué pasa	Corrección
`intercambiar(x, y);`	Advertencia (error en gcc 14+)	`intercambiar(&x, &y);`
`*cantidad++;`	No aumenta el valor	`(*cantidad)++;`
`void f(int x)` para cambiar x	Cambia una copia	`void f(int *x)`
`int *p; *p = 5;`	Puntero sin dirección	`int *p = &v;`

Glosario

Término	Definición breve
Dirección	Ubicación de una variable en la memoria; se obtiene con &.
Paso por dirección	La función recibe la dirección y puede modificar la variable original.
Paso por valor	La función recibe una copia del dato.
Puntero	Variable que guarda una dirección de memoria.
const	En un parámetro, impide que la función lo modifique.
Desreferenciar	Acceder al valor de una dirección con *.
Programa modular	Programa organizado en funciones con una tarea cada una.

Preguntas de repaso

1. ¿Qué recibe una función por valor?
2. ¿Qué hace &x?
3. ¿Qué significa *p = 21?
4. ¿Cómo se devuelven dos resultados?
5. ¿Por qué un arreglo se pasa sin &?
6. ¿Qué diferencia hay entre *p++ y (*p)++?

Fuentes recomendadas

- Kernighan, B. y Ritchie, D. (1991). El lenguaje de programación C (2.ª ed.). Prentice Hall.
- cppreference.com: punteros, paso de argumentos y arreglos como parámetros.
- Algoritmos de PSeInt de la UA1 (sesiones 6 y 9) y programas de las sesiones 19 a 21.